Лекция 6. Особые точки и локальные дескрипторы

Детектор Харриса, масштабное пространство, SIFT, сопоставление и ratio test, оценка качества

Александр Савельев · Курс «Анализ изображений», 3 курс бакалавриата

Повторение: что нужно из лекций 3–5

- **Градиент** $\nabla I = (I_x, I_y)$: модуль — сила перепада, направление — куда растёт яркость (лекция 5).
- **Производная гауссиана**: $I_x = \partial_x G_\sigma * I$ — устойчивая к шуму производная; σ — масштаб (лекция 5).
- **LoG и DoG**: $G_{k\sigma} - G_\sigma \approx (k-1)\sigma^2 \nabla^2 G_\sigma$; отклик максимален на пятнах (лекция 5).
- **Пирамида Гаусса**: размыть и уменьшить в 2 раза (лекция 4).
- **Подавление немаксимумов**: оставить только локальные максимумы отклика (лекция 5).

Сегодня: от границ к точкам

Граница — линия, вдоль неё все пиксели похожи, и точку на ней не найти повторно. Сегодня ищем места, которые **узнаются однозначно** на другом снимке той же сцены.

Главный вопрос лекции

Как найти одни и те же точки на двух снимках, если камера повернулась, приблизилась и свет изменился?

План лекции

1. **Зачем особые точки** — задача сопоставления и требования — 7 мин
2. **Углы: Харрис и Ши–Томаси** — структурный тензор и его собственные числа — 20 мин
3. **Масштабное пространство** — автоматический выбор масштаба — 12 мин
4. **SIFT по шагам** — детектор и дескриптор, инвариантный к масштабу и повороту — 31 мин
5. **Сопоставление и оценка** — ratio test, поиск соседей, метрики — 20 мин

Сквозная линия

Хорошая точка — та, у которой окрестность **меняется при любом сдвиге**. Её **масштаб** и **ориентацию** определяем по самому изображению, затем описываем окрестность вектором-**дескриптором** и ищем самый близкий вектор на другом снимке.

Часть 1

Зачем особые точки

О чём эта часть

Определение

Особая (ключевая) точка (keypoint, interest point) — точка изображения с координатами (x, y) и обычно масштабом σ и углом θ, которую можно **надёжно найти снова** на другом снимке той же сцены.

Дескриптор — вектор чисел, описывающий окрестность точки; похожие окрестности — **близкие векторы**.

Сопоставление (matching) — поиск пар точек двух снимков, изображающих одно и то же место сцены.

Что покажем

- где нужно сопоставление;
- конвейер: детектор $\rightarrow$ дескриптор $\rightarrow$ сопоставление;
- четыре требования к хорошим точкам.

Задача сопоставления

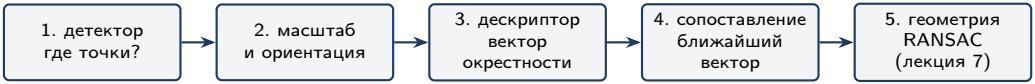
одна и та же сцена с двух ракурсов: 70 найденных соответствий точек



Граффити с двух ракурсов: поворот, сжатие, другой свет. Линии соединяют точки, которые алгоритм (SIFT) счёл одинаковыми; все 70 показанных — верные (проверено по известной гомографии).

Где нужно: склейка панорам (лекция 7), 3D-реконструкция по фото, трекинг объектов в видео, поиск объекта или документа на снимке, визуальная локализация робота. В KYC — сравнение документа с шаблоном.

Конвейер и требования к точкам



Четыре требования

1. **Повторяемость**: та же точка сцены находится на обоих снимках, несмотря на поворот, масштаб, свет.
2. **Различимость**: окрестность не похожа на окрестности других точек — иначе пару не выбрать.
3. **Локальность**: окрестность маленькая — устойчиво к заслонениям и перспективе.
4. **Эффективность**: сотни–тысячи точек за миллисекунды.

Почему не все пиксели. Пиксель на ровной стене похож на тысячи соседей, на границе — на всех соседей вдоль неё. Однозначно узнаются только места, где яркость меняется **во всех направлениях**: углы, пятна, перекрестья.

Инвариантность (не меняется при преобразовании) и **ковариантность** (меняется вместе с ним): координаты точки должны **двигаться вместе со сценой**, а дескриптор — **оставаться тем же**.

Часть 2

Углы: Харрис и Ши–Томаси

О чём эта часть

Определение

Угол — точка, в окрестности которой яркость меняется **в двух разных направлениях** (пересечение двух границ).

Структурный тензор M — матрица 2×2 из усреднённых произведений производных $I_x^2,\ I_x I_y,\ I_y^2$ в окне. Её собственные числа $\lambda_1 \geq \lambda_2$ показывают, насколько сильно меняется яркость в самом «крутом» и в самом «пологом» направлении.

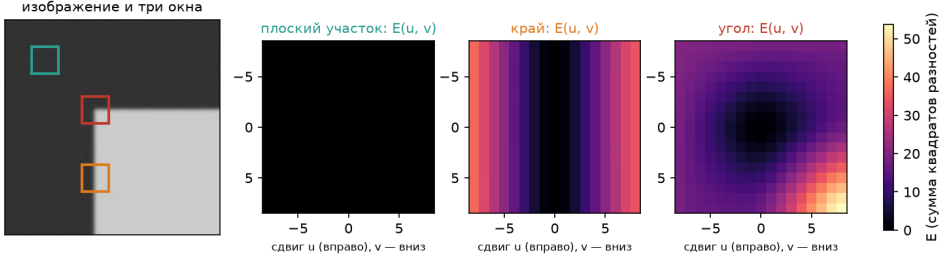
Что покажем

- окно, которое сдвигаем;
- как из сдвига получается матрица M;
- что значат λ_1, λ_2;
- отклики Харриса и Ши–Томаси;
- к чему угол инвариантен, а к чему нет.

Идея: сдвинуть окно

$$E(u,v) = \sum_{(x,y)\in W} w(x,y)\,\big[I(x+u,\ y+v) - I(x,y)\big]^2$$

- W — окно вокруг точки; $w(x,y)$ — вес пикселя в окне (1 внутри или гауссиана);
- (u,v) — сдвиг окна по горизонтали и вертикали; E — насколько **окно после сдвига не похоже** на исходное.



Плоский участок: $E \approx 0$ при любом сдвиге. **Край**: E растёт только поперёк края, вдоль — ноль (тёмная полоса). **Угол**: E растёт **в любом** направлении — только такое окно можно однозначно найти снова.

От сдвига к матрице M

$$I(x+u, y+v) \approx I + I_x u + I_y v \quad \Longrightarrow \quad E(u,v) \approx \sum w\,(I_x u + I_y v)^2 = \begin{pmatrix} u & v \end{pmatrix} M \begin{pmatrix} u \\ v \end{pmatrix}$$

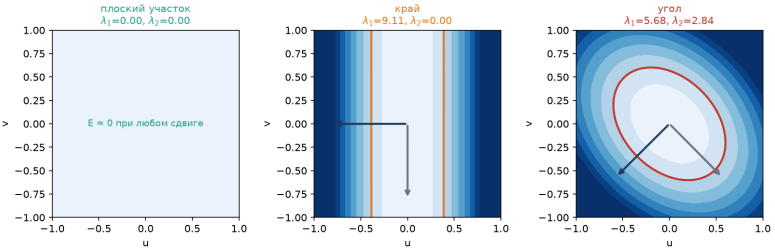
$$M = \sum_{(x,y)\in W} w(x,y) \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix}$$

- первая формула — ряд Тейлора первого порядка: при малом сдвиге яркость меняется на $\nabla I \cdot (u, v)$;
- $(I_x u + I_y v)^2 = I_x^2 u^2 + 2 I_x I_y\, uv + I_y^2 v^2$ — квадратичная форма; её коэффициенты, **усреднённые по окну**, и составляют M;
- I_x, I_y — производные гауссиана с малым σ_D (масштаб дифференцирования); сумма с весом w — гауссово сглаживание произведений с σ_I (масштаб интегрирования, обычно $\sigma_I \approx 2\sigma_D$).

Почему важно

Вместо перебора всех сдвигов (u, v) достаточно **трёх чисел на пиксель**: $\langle I_x^2 \rangle$, $\langle I_x I_y \rangle$, $\langle I_y^2 \rangle$. Вся информация о том, «как меняется окно при сдвиге», сидит в матрице 2×2.

Собственные числа M



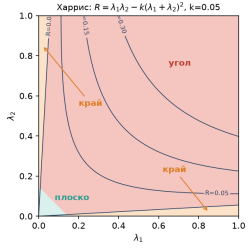
$$M = Q \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix} Q^\top, \quad \lambda_1 \geq \lambda_2 \geq 0$$

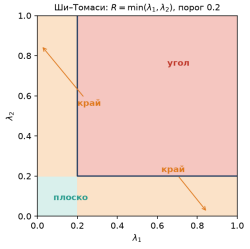
- столбцы Q — два перпендикулярных направления (стрелки на рисунке);
- λ_1 — рост E вдоль самого «крутого» направления, λ_2 — вдоль самого «пологого».

Линия уровня $E(u, v) = \text{const}$ — **эллипс** с полуосями $\sim 1/\sqrt{\lambda}$.

- обе λ малы — **плоско**, эллипса нет;
- $\lambda_1 \gg \lambda_2 \approx 0$ — **край**: эллипс вырождается в две прямые;
- обе λ большие — **угол**: маленький замкнутый эллипс.

Отклики Харриса и Ши–Томаси





$$R_{\mathrm{H}} = \det M - k\,(\operatorname{tr} M)^2 = \lambda_1\lambda_2 - k(\lambda_1+\lambda_2)^2$$

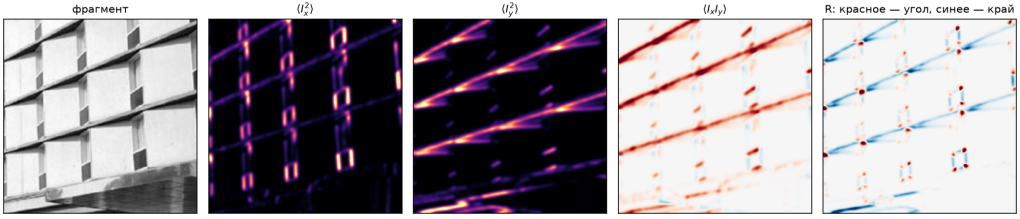
- $\det M = \lambda_1\lambda_2 = M_{11}M_{22} - M_{12}^2$, $\operatorname{tr} M = \lambda_1 + \lambda_2 = M_{11} + M_{22}$;
- $k \approx 0.04$–0.06: **штраф за вытянутость**; $R < 0$ на крае, $R \gg 0$ на угле.

Харрис и Стивенс (1988) выбрали det и tr, чтобы **не считать** собственные числа: всё через три элемента M.

$$R_{\mathrm{ST}} = \min(\lambda_1, \lambda_2)$$

- Ши и Томаси (1994): угол хорош, если даже **самое слабое** направление меняется заметно; $\lambda_2 = \frac{\mathrm{tr}}{2} - \sqrt{\frac{\mathrm{tr}^2}{4} - \det}$.

Матрица M на снимке



$\langle I_x^2 \rangle$ светится на **вертикальных** границах окон, $\langle I_y^2 \rangle$ — на **наклонных горизонтальных** (перспектива фасада). $\langle I_x I_y \rangle$ — знак наклона границы: красное и синее для двух диагоналей.

R **положителен** (красный) только там, где есть **обе** компоненты — в углах рам. На прямых участках границ $R < 0$ (синий): один большой λ, второй ≈ 0.

Детектор Харриса: алгоритм

Алгоритм

Вход: I, масштабы σ_D, σ_I, k, порог t.
Выход: список углов (x, y).

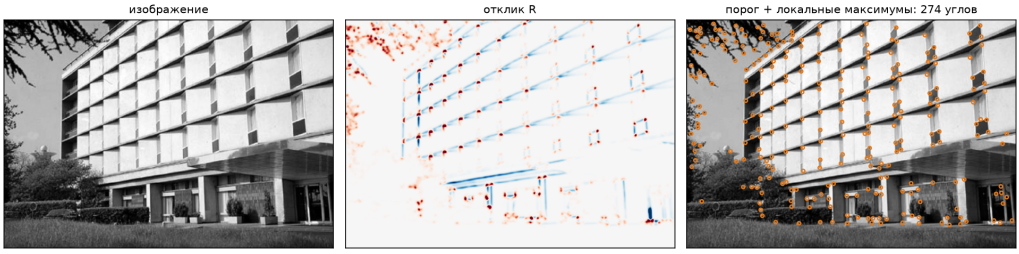
1. I_x, I_y — производные гауссиана с σ_D
2. $S_{xx} = G_{\sigma_I} * I_x^2$, $S_{yy} = G_{\sigma_I} * I_y^2$, $S_{xy} = G_{\sigma_I} * (I_x I_y)$
3. $R = S_{xx} S_{yy} - S_{xy}^2 - k(S_{xx} + S_{yy})^2$
4. оставить пиксели с $R > t$ (часто $t = 0.01 \max R$)
5. **подавление немаксимумов** в окне 3×3 или больше

Почему так. Шаг 2 — это сумма по окну w из определения M, сделанная **сразу для всех пикселей** одной свёрткой. Шаг 5 — как в Кэнни: вокруг угла R большой у нескольких соседей, оставляем один.

Стоимость: 5 гауссовых свёрток (разделимых) и арифметика — миллисекунды на снимок.

OpenCV: `cv2.cornerHarris` (карта R) и `cv2.goodFeaturesToTrack` — Ши–Томаси по умолчанию или Харрис с `useHarrisDetector=True`, сразу с порогом и минимальным расстоянием между точками.

Харрис на снимке



Карта R: красные точки — углы рам и стыков, синие полосы — прямые участки границ ($R < 0$). Почти везде $R \approx 0$: плоские участки стены и неба.

После порога $0.02 \max R$ и подавления немаксимумов в радиусе 6 пикселей — 274 угла. Они **плотно сидят** на окнах и деревьях и почти отсутствуют на ровной стене — как и требуется.

К чему инвариантен Харрис

Поворот — да. При повороте картинки на R_α градиент поворачивается, а $M' = R_\alpha M R_\alpha^\top$ — у неё **те же собственные числа**. Угол остаётся углом.

Сдвиг яркости $I + b$ — да: производные не меняются.

Контраст aI — частично: R умножается на a^4, поэтому меняется число точек выше порога t.

Масштаб — нет. Окно фиксированного размера на крупном плане видит дугу как край, на мелком — как угол.



Одна и та же дуга: в маленьком окне ($\sigma_I = 2$) отклик всюду отрицательный — «край», в большом ($\sigma_I = 40$) — положительный «угол». **Правильный размер окна зависит от масштаба структуры** — следующая часть.

Листинг: Харрис в NumPy

```
from scipy import ndimage as ndi
from skimage.feature import peak_local_max

def harris(I, sd=1.0, si=2.0, k=0.05):
    Ix = ndi.gaussian_filter(I, sd, order=(0, 1))
    Iy = ndi.gaussian_filter(I, sd, order=(1, 0))
    # окно w: гауссово усреднение
    Sxx = ndi.gaussian_filter(Ix * Ix, si)
    Syy = ndi.gaussian_filter(Iy * Iy, si)
    Sxy = ndi.gaussian_filter(Ix * Iy, si)
    det = Sxx * Syy - Sxy ** 2
    tr = Sxx + Syy
    return det - k * tr ** 2

R = harris(I)
pts = peak_local_max(R, min_distance=6,
                     threshold_rel=0.02)
```

- `I` — `float` в $[0, 1]$.
- `sd` — σ_D (производные), `si` — σ_I (окно); строки 7–9 — три элемента M **во всех пикселях сразу**.
- Строки 10–12 — $\det M - k\,\mathrm{tr}^2 M$ без собственных чисел.
- `peak_local_max`: порог $0.02 \max R$ и подавление немаксимумов в радиусе 6 пикселей; результат — массив (строка, столбец).
- Для Ши–Томаси вместо строки 12: `tr / 2 - np.sqrt(tr**2 / 4 - det)`.

Часть 3

Масштабное пространство

О чём эта часть

Определение

Масштабное пространство — семейство всё сильнее размытых копий изображения:

$$L(x, y, \sigma) = G_\sigma * I, \quad \sigma \geq 0$$

Третья координата σ — «насколько крупные детали ещё видны».

Характерный масштаб точки — значение σ, при котором отклик детектора в ней **максимален по** σ.

Что покажем

- как выглядит масштабное пространство;
- зачем нормировать производные на σ;
- выбор масштаба пятна по LoG;
- связь с DoG.

Одна картинка — все масштабы



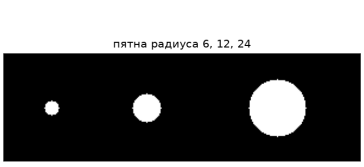
При росте σ исчезают сначала мелкие детали (трава, буквы), потом средние (камера, лицо), остаются крупные пятна. **Гаусс — единственное** линейное сглаживание, которое не создаёт новых деталей при увеличении σ (Koenderink, 1984; Lindeberg, 1994).

Зачем. Если снимок уменьшить в 2 раза, структура радиуса r станет радиуса $r/2$. Её отклик переедет с уровня σ на уровень $\sigma/2$. Если искать экстремумы **и по** σ, найденный масштаб изменится вместе со снимком — это и есть **ковариантность к масштабу**.

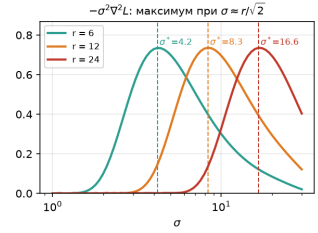
Нормировка по масштабу

$$\nabla^2_{\text{norm}} L = \sigma^2 \nabla^2 L = \sigma^2 \left(L_{xx} + L_{yy}\right)$$

- L_{xx}, L_{yy} — вторые производные размытого изображения $L(\cdot, \cdot, \sigma)$;
- множитель σ^2 — по одному σ **на каждую производную** (Lindeberg, 1998).



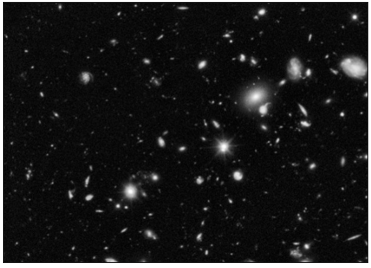




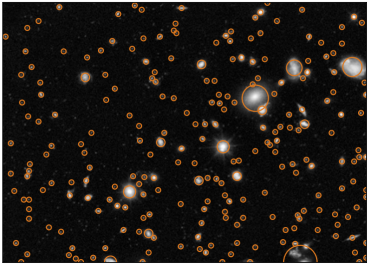
Почему σ^2. Размытие уменьшает амплитуду производных: перепад размазан на ширину $\sim \sigma$, значит, первая производная $\sim 1/\sigma$, вторая $\sim 1/\sigma^2$. Без нормировки отклик просто падает с ростом σ и крупные пятна не выигрывают (средний график). С σ^2 у каждого пятна — **свой максимум** при $\sigma^* \approx r/\sqrt{2}$: 4.2, 8.3, 16.6 для $r = 6, 12, 24$ (теория: 4.24, 8.49, 16.97).

Детектор пятен

снимок Хаббла (фрагмент)



экстремумы $\sigma^2\nabla^2 L$ по (x, y, σ): 236 пятен, радиус $\sqrt{2}\sigma$



Алгоритм. Посчитать $\sigma^2\nabla^2 L$ для ряда σ (стопка изображений), найти точки, где значение **больше всех соседей** и по (x, y), и по σ. Каждая найденная точка — пятно с центром (x, y) и радиусом $r = \sqrt{2}\sigma$.

На снимке Хаббла найдено 236 галактик и звёзд разного размера — **без** подбора размера окна руками (`skimage.feature.blob_log`).

DoG вместо LoG:
$G_{k\sigma} * I - G_\sigma * I \approx (k-1)\,\sigma^2\nabla^2 G_\sigma * I$ (лекция 5) — в DoG **нормировка σ^2 уже встроена**. Это идея SIFT.

Часть 4

SIFT по шагам

О чём эта часть

Определение

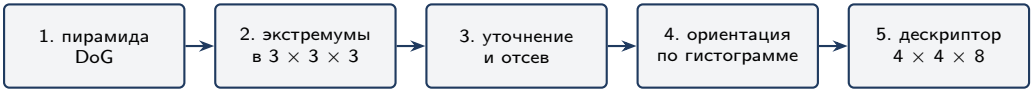
SIFT (Scale-Invariant Feature Transform, D. Lowe, 1999/2004) — детектор и дескриптор: находит пятна в пирамиде DoG, задаёт каждой точке **масштаб и ориентацию** и описывает её окрестность вектором из **128 чисел** — гистограммами направлений градиента.

Патент истёк в 2020 году; SIFT входит в основной OpenCV (`cv2.SIFT_create`).

Что покажем

- пять шагов SIFT;
- пирамиду DoG и поиск экстремумов;
- уточнение и отсев слабых точек и краёв;
- ориентацию и дескриптор $4 \times 4 \times 8$;
- какой шаг даёт какую инвариантность.

SIFT: пять шагов



Шаги 1–3 — детектор. Находят точку и её масштаб: **где** и **какого размера** окрестность описывать.

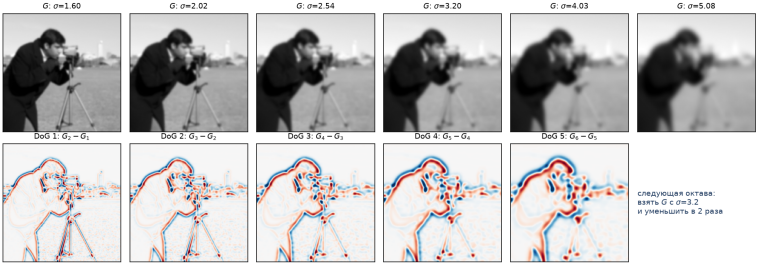
Шаги 4–5 — дескриптор. Поворачивают окрестность к «своему» направлению и превращают её в вектор.

Главная идея. Окрестность берётся в **собственной системе координат точки**: центр (x, y), единица длины σ, ось — θ. На любом снимке той же сцены эта система «переезжает» вместе с объектом, поэтому описание совпадает.

Шаг 1: пирамида DoG

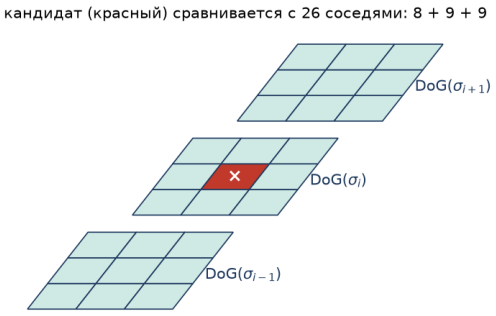
$$D(x, y, \sigma_i) = L(x, y, \sigma_{i+1}) - L(x, y, \sigma_i), \qquad \sigma_i = \sigma_0\, k^i, \quad k = 2^{1/s}$$

- $L = G_\sigma * I$; $\sigma_0 = 1.6$; $s = 3$ — число интервалов в **октаве** (удвоении σ), $k = 2^{1/3} \approx 1.26$;
- в октаве $s + 3 = 6$ размытий и 5 DoG; экстремумы ищут в средних $s = 3$, чтобы у каждого были соседи сверху и снизу.



Следующая октава: размытие с $\sigma = 2\sigma_0$ уменьшают в 2 раза (пирамида из лекции 4) и повторяют — работа на каждой октаве в 4 раза меньше. DoG ярко отвечает на пятна и границы своего масштаба: при росте σ полосы толще.

Шаг 2: экстремумы $3 \times 3 \times 3$



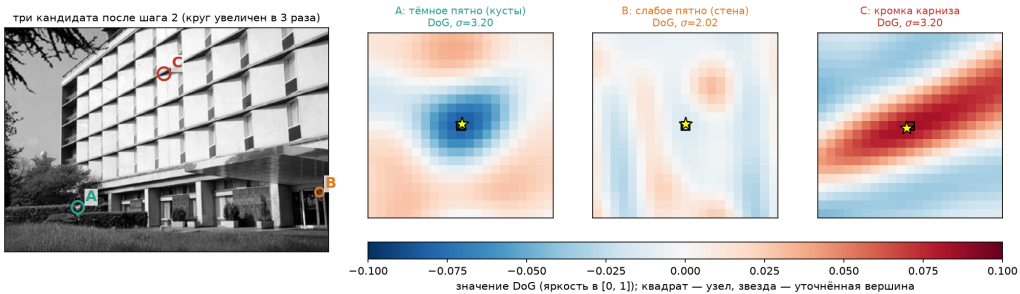
Точка (x, y, σ_i) — кандидат, если D в ней **больше всех 26 соседей** или **меньше всех**: 8 на своём уровне, по 9 на соседних уровнях масштаба.

Почему по трём координатам. Максимум по (x, y) даёт центр пятна, максимум по σ — его размер. Оба меняются вместе со снимком.

Почему быстро. Большинство пикселей отсеиваются после первых нескольких сравнений. На снимке 640×442 — около 7 600 кандидатов из почти 300 000 пикселей на уровень.

Максимум — светлое пятно на тёмном фоне, минимум — тёмное на светлом.

Сквозной пример: три кандидата

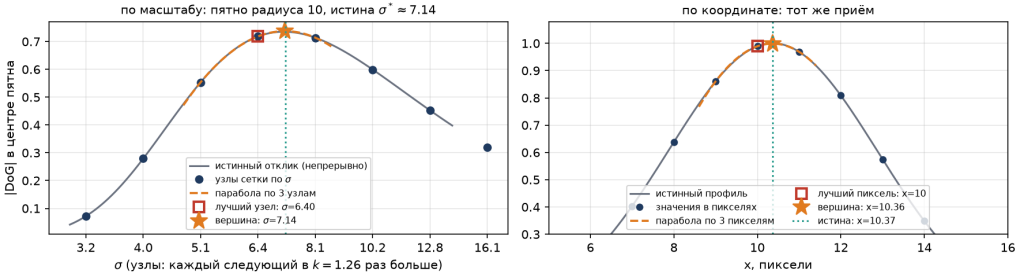


Строим пирамиду DoG для фасада (640×442, первая октава, $\sigma_0 = 1.6$, $s = 3$). Шаг 2 даёт **851 кандидат**. Проведём три из них через шаги 3а и 3б — с настоящими числами.

OpenCV считает все октавы и удваивает снимок, поэтому точек у него больше (слайд «Результат отсева»), но правила те же.

	узел (x, y)	σ	D в узле
A	$(144, 354)$	2.54	-0.084 (минимум)
B	$(622, 324)$	2.02	-0.015 (минимум)
C	$(315, 90)$	3.20	$+0.085$ (максимум)

Шаг 3a: экстремум лежит между узлами сетки



Что не так после шага 2. Мы знаем только **лучший узел сетки**, а настоящий максимум обычно между узлами. Сетка по σ грубая: каждый следующий уровень в 1.26 раза больше. На верхних октавах один пиксель — это 2, 4, 8 пикселей исходного снимка.

Пример слева. Пятно радиуса 10, истинный масштаб $\sigma^* = 7.14$. Лучший узел — $\sigma = 6.40$, ошибка 10 %.

Идея. Провести **параболу через три соседних узла** и взять её вершину: $\sigma = 7.14$, ошибка меньше 1 %. Справа — то же по координате x: $10 \to 10.36$ при истине 10.37.

Шаг 3а: вершина параболы по трём точкам

$$D(t) \approx D_0 + D'\,t + \tfrac{1}{2}D''\,t^2, \qquad D' = \frac{D_+ - D_-}{2}, \quad D'' = D_+ - 2D_0 + D_-, \qquad \hat{t} = -\frac{D'}{D''}$$

- t — сдвиг от лучшего узла в шагах сетки: $t = -1$ и $t = +1$ — соседние узлы;
- D_-, D_0, D_+ — значения DoG в левом соседе, в узле и в правом соседе;
- D' — наклон (центральная разность из лекции 5), D'' — кривизна (у максимума $D'' < 0$);
- $\hat{t}$ — вершина: там производная параболы $D' + D''\,t$ равна нулю.

Численный пример (по σ, рисунок слева)

$D_- = 0.553$ ($\sigma = 5.08$), $D_0 = 0.718$ ($\sigma = 6.40$),
$D_+ = 0.714$ ($\sigma = 8.06$)

$D' = \frac{0.714 - 0.553}{2} = 0.080$,
$D'' = 0.714 - 1.436 + 0.553 = -0.169$

$\hat{t} = -\frac{0.080}{-0.169} = 0.47$ шага $\Rightarrow$ $\hat{\sigma} = 6.40 \cdot 1.26^{0.47} = \mathbf{7.14}$

Как читать ответ. D_+ почти равно D_0 — значит, вершина почти посередине между ними: $\hat{t} \approx 0.5$.

Правило. Если $|\hat{t}| > 0.5$, вершина ближе к соседнему узлу: переходим в него и считаем заново.

Шаг 3а: та же парабола сразу по x, y, σ

$$D(\mathrm{x}_0 + \boldsymbol{\delta}) \approx D + \nabla D^{\top}\boldsymbol{\delta} + \tfrac{1}{2}\,\boldsymbol{\delta}^{\top} H_D\,\boldsymbol{\delta} \quad\Longrightarrow\quad \hat{\boldsymbol{\delta}} = -H_D^{-1}\,\nabla D$$

- $\mathrm{x}_0 = (x, y, \sigma)$ — найденный узел; $\boldsymbol{\delta} = (\delta_x, \delta_y, \delta_\sigma)$ — поправка, **три числа t** вместо одного;
- $\nabla D = (D_x, D_y, D_\sigma)$ — три наклона, каждый как D'; H_D (3×3) — на диагонали три кривизны, как D'', вне диагонали — смешанные D_{xy}, $D_{x\sigma}$, $D_{y\sigma}$;
- $\hat{\boldsymbol{\delta}} = -H_D^{-1}\nabla D$ — то же, что $\hat{t} = -D'/D''$: деление заменяется решением системы 3×3.

Алгоритм уточнения

Вход: узел (x, y, i) — кандидат из шага 2.
Выход: $(\hat{x}, \hat{y}, \hat{\sigma})$ или «отброшен».

1. ∇D и H_D — разностями по 26 соседям
2. решить $H_D\,\boldsymbol{\delta} = -\nabla D$
3. если какое-то $|\delta_j| > 0.5$ — сдвинуться в соседний узел и вернуться к п. 1 (не больше 5 раз)
4. иначе $\hat{\mathrm{x}} = \mathrm{x}_0 + \hat{\boldsymbol{\delta}}$; не сошлось — отбросить

Почему 3D, а не три 1D

Если бы смешанные производные были нулями, H_D была бы диагональной и решение распалось бы на **три отдельные параболы**. У наклонного или вытянутого пятна они не нули: сдвиг по x меняет лучший σ. Система 3×3 учитывает это сразу.

Пример: уточняем кандидата A

Числа в единицах 10^{-3}; сдвиг по σ — в уровнях пирамиды (один уровень — $\times 1.26$).

Итерация 1: узел $(144, 354)$, $\sigma = 2.54$

$\nabla D = (0.33,\ 0.28,\ -4.66)$

$$H_D = \begin{pmatrix} 5.39 & 2.96 & -2.24 \\ 2.96 & 12.0 & 2.04 \\ -2.24 & 2.04 & 9.71 \end{pmatrix}$$

$\hat{\boldsymbol{\delta}} = -H_D^{-1}\nabla D = (0.29,\ -0.19,\ 0.59)$

$|\delta_\sigma| = 0.59 > 0.5 \Rightarrow$ вершина ближе к **уровню выше**: переходим в узел с $\sigma = 3.20$.

Итерация 2: узел $(144, 354)$, $\sigma = 3.20$

$\nabla D = (-1.04,\ 1.64,\ 5.59)$

$\hat{\boldsymbol{\delta}} = (0.17,\ -0.16,\ -0.50)$ — все ≤ 0.5, **стоп**.

$\hat{x} = 144.17$, $\hat{y} = 353.84$,
$\hat{\sigma} = 1.6 \cdot 1.26^{3-0.5} = 2.85$

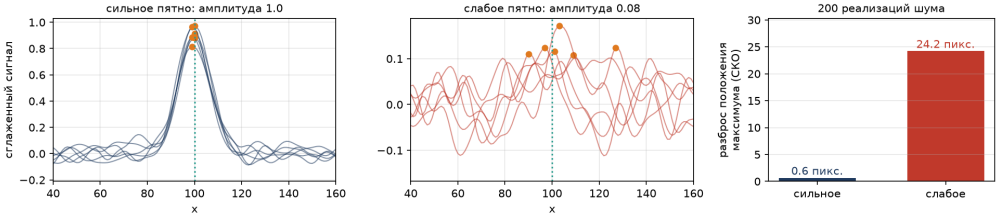
Что видно. По x, y поправка меньше пикселя, а по масштабу вершина оказалась **ровно между** уровнями 2.54 и 3.20 — шаг 2 сам по себе этого не видит.

B и C сходятся за одну итерацию:
$\hat{\boldsymbol{\delta}}_B = (0.04, -0.14, -0.09)$,
$\hat{\boldsymbol{\delta}}_C = (-0.37, 0.33, -0.37)$. Из 851 кандидата сошлись 650, остальные ушли за край или не сошлись за 5 итераций.

Шаг 3a: отсев слабого контраста

$$D(\hat{\mathrm{x}}) = D(\mathrm{x}_0) + \tfrac{1}{2}\,\nabla D^{\top}\hat{\boldsymbol{\delta}}, \qquad \text{отбросить, если } |D(\hat{\mathrm{x}})| < 0.03$$

- $D(\hat{\mathrm{x}})$ — **высота пика** в уточнённой вершине (подставили $\hat{\boldsymbol{\delta}}$ в параболоид); это и есть контраст точки;
- 0.03 — порог Лоу для яркости в $[0, 1]$ и $s = 3$ уровней в октаве.



Зачем. Один и тот же сигнал, шум 0.15, 200 повторов. Сильное пятно: максимум всегда в одном месте (разброс **0.6 пикселя**). Слабое: максимум прыгает на **24 пикселя** — его положение задаёт шум, а не сцена. На другом снимке такая точка не найдётся, описывать её бесполезно. На фасаде так уходит половина кандидатов: 7 654 → 3 657.

Пример: контраст A, B, C

$$D(\hat{\mathbf{x}}) = D(\mathbf{x}_0) + \tfrac{1}{2}\,\nabla D^{\top}\hat{\boldsymbol{\delta}}$$

Кандидат A (последний узел)

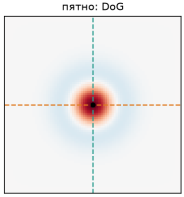
$$D(\hat{\mathbf{x}}) = -0.0835 + \tfrac{1}{2}\big(-1.04\cdot 0.17 + 1.64\cdot(-0.16) + 5.59\cdot(-0.50)\big)\cdot 10^{-3} = -0.0835 - 0.0016 = \mathbf{-0.085}$$

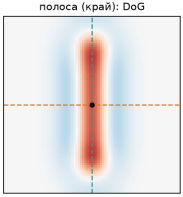
	D в узле	поправка $\frac{1}{2}\nabla D^{\top}\hat{\boldsymbol{\delta}}$	$\lvert D(\hat{\mathbf{x}})\rvert$	сравнение с 0.03
A	-0.0835	-0.0016	0.085	проходит
B	-0.0150	≈ 0	0.015	отброшен: слабый
C	$+0.0845$	$+0.0004$	0.085	проходит

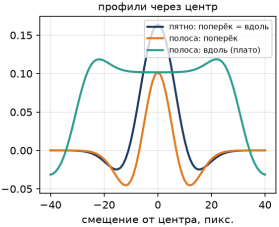
Что видно. Поправка меняет высоту пика всего на 1–2 %: главное здесь — сравнение с порогом. B — едва заметное пятнышко на стене: его высота в 2 раза ниже порога, на другом снимке шум сдвинет или сотрёт его. Из 650 сошедшихся кандидатов по контрасту отброшены 219.

C прошёл — хотя лежит на кромке. Контраст его не ловит: это работа шага 3б.

Шаг 3б: почему точки на краях плохие









Пятно. Отклик DoG убывает **во все стороны** (синий профиль): максимум один, его положение однозначно.

Полоса. Поперёк — острый пик (оранжевый), вдоль — **плато** (бирюзовый): на всём отрезке DoG почти одинаков. «Максимумом» может оказаться любой пиксель отрезка, выберет его шум.

Почему шаг 3а не помог. Такая точка контрастная и формально экстремум среди 26 соседей. Но на втором снимке она окажется в **другом месте** того же края.

Справа: 791 точка, отброшенная как край на фасаде: они лежат вдоль рам и карнизов.

Шаг 3б: кривизна и гессиан

Определение

Кривизна профиля — его вторая производная: насколько быстро он загибается. У острого пика кривизна большая, у плато — около нуля.

$$H = \begin{pmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{pmatrix}$$

- H — **гессиан**: все вторые производные D по x и y; $D_{xx} = D[x{+}1] - 2D + D[x{-}1]$ — как D'' в шаге 3а;
- собственные векторы H — **самое крутое и самое пологое** направления;
- собственные числа $\lambda_1 \geq \lambda_2$ (по модулю) — кривизны вдоль них.

Где (рис.)	$\lvert\lambda_1\rvert$	$\lvert\lambda_2\rvert$	вывод
пятно	0.0049	0.0049	оставить
полоса	0.0063	≈ 0	отбросить
угол	большое	большое	оставить

Нужна **не величина**, а **отношение** $r = \lambda_1/\lambda_2$: у пятна $r = 1$, у края $r \to \infty$.

Как у Харриса

Та же логика, что со структурным тензором M: «оба собственных числа большие» — хорошая точка, «одно» — край. Только здесь матрица из вторых производных DoG, а не из градиентов.

Шаг 3б: проверка без собственных чисел

1. Две величины без корней:

$$\operatorname{tr} H = D_{xx} + D_{yy} = \lambda_1 + \lambda_2$$
$$\det H = D_{xx} D_{yy} - D_{xy}^2 = \lambda_1 \lambda_2$$

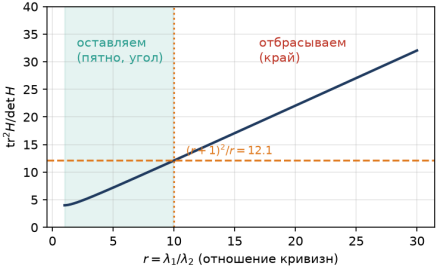
2. Подставим $\lambda_1 = r\,\lambda_2$**:**

$$\frac{\operatorname{tr}^2 H}{\det H} = \frac{(r\lambda_2 + \lambda_2)^2}{r\lambda_2^2} = \frac{(r+1)^2}{r}$$

λ_2 сократилось: дробь зависит **только от отношения** кривизн и растёт вместе с r.

3. Правило ($r = 10$):

$$\det H > 0 \quad \text{и} \quad \frac{\operatorname{tr}^2 H}{\det H} < \frac{(10+1)^2}{10} = 12.1$$



Примеры:

- пятно: $r = 1$, дробь $= 4 < 12.1$ — оставить;
- полоса: $\lambda_2 \approx 0$, $\det H \approx 0$, дробь огромна — отбросить;
- $\det H < 0$ — кривизны разных знаков (седло), не пик — отбросить.

Пример: тест краёв для A и C

Вторые производные по x, y в последнем узле, единицы 10^{-3} (для det — 10^{-6}).

A: тёмное пятно

$D_{xx} = 5.71$, $D_{yy} = 9.04$, $D_{xy} = 1.42$

$\operatorname{tr} H = 5.71 + 9.04 = 14.75$

$\det H = 5.71 \cdot 9.04 - 1.42^2 = 49.6 > 0$

$$\frac{\operatorname{tr}^2 H}{\det H} = \frac{217.6}{49.6} = 4.4 < 12.1$$ — **оставить**

Кривизны 9.6 и 5.2: $r = 1.8$ — почти круглый пик.

C: кромка карниза

$D_{xx} = -2.10$, $D_{yy} = -11.39$, $D_{xy} = -3.96$

$\operatorname{tr} H = -13.49$

$\det H = 23.9 - 15.7 = 8.2 > 0$

$$\frac{\operatorname{tr}^2 H}{\det H} = \frac{182.0}{8.2} = 22 > 12.1$$ — **отбросить**

Кривизны -12.9 и -0.6: $r = 20$ — поперёк кромки пик, вдоль — почти плато.

Итог примера	кандидаты (шаг 2)	сошлись	после контраста	после краёв
все точки октавы	851	650	431	**237**
A / B / C	A, B, C	A, B, C	A, C	**A**

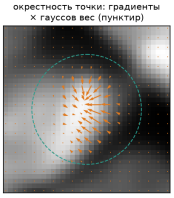
Результат отсева



Все экстремумы (7 654): сплошной ковёр на траве, кустах, ровной стене — случайные колебания шума тоже дают экстремумы DoG.
После контраста (3 657): ушли точки в однородных областях и на траве с мелкой текстурой.

После отсева краёв (2 866): ушли точки, сидящие на длинных прямых границах фасада. Остались углы рам, концы элементов, пятна — **хорошо локализованные** места.
OpenCV: `contrastThreshold=0.04` (делится на число уровней в октаве), `edgeThreshold=10`.

Шаг 4: доминирующая ориентация







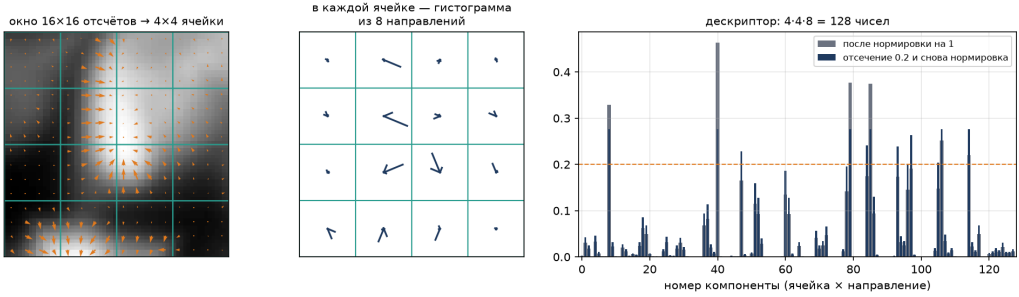
Алгоритм

1. в окрестности точки на её уровне $L(\cdot, \sigma)$ посчитать модуль m и угол θ градиента;
2. вес пикселя $= m \cdot$ гауссиана с 1.5σ от центра;
3. гистограмма углов: 36 корзин по 10°;
4. пик — ориентация точки (уточняют параболой по 3 корзинам).

Зачем. Дескриптор будет считаться в окне, **повёрнутом на θ**. Если снимок повернут на 30°, θ тоже вырастет на 30° — окно повернётся вместе с объектом, и описание совпадёт.

Несколько пиков. Пик выше **80 %** от главного рождает **ещё одну точку** с той же позицией и своей ориентацией (около 15 % точек).

Шаг 5: дескриптор $4 \times 4 \times 8 = 128$



Окно. 16×16 отсчётов вокруг точки, **повёрнутое на θ** и растянутое пропорционально σ, делят на 4×4 ячейки по 4×4 отсчёта.

Ячейка. Гистограмма направлений градиента на **8 корзин** по 45°; вес отсчёта — модуль градиента $\times$ гауссиана (половина ширины окна).

Вектор. 16 гистограмм по 8 корзин подряд — 128 чисел (правый график).

Интерполяция. Вклад каждого отсчёта **делится между соседними** ячейками и корзинами (трилинейно): малый сдвиг или поворот не перекидывает его целиком в другую корзину.

Шаг 5: нормировка дескриптора

$$\mathrm{v} \leftarrow \frac{\mathrm{v}}{\|\mathrm{v}\|}, \qquad v_j \leftarrow \min(v_j,\ 0.2), \qquad \mathrm{v} \leftarrow \frac{\mathrm{v}}{\|\mathrm{v}\|}$$

- $\mathrm{v} \in \mathbb{R}^{128}$ — склеенные 16 гистограмм по 8 корзин; $\|\mathrm{v}\|$ — евклидова длина;
- v_j — одна компонента; 0.2 — порог отсечения, подобранный Лоу на данных.

Первая нормировка — против контраста

При $I \to aI$ все градиенты, а значит, все v_j умножаются на a. После деления на длину $\frac{a\mathrm{v}}{a\|\mathrm{v}\|} = \frac{\mathrm{v}}{\|\mathrm{v}\|}$ — **a сокращается**. Сдвиг $I + b$ градиенты не меняет вообще.

Отсечение — против бликов

Блик, пересвет, тень на части окна дают **несколько огромных** градиентов. Без отсечения они доминируют в векторе. После $\min(v_j, 0.2)$ и повторной нормировки важнее становится **распределение направлений**, а не величина отдельных перепадов.

Точки SIFT и их инвариантность

точки SIFT: круг — масштаб, отрезок — ориентация



70 самых сильных точек: **круг** — область дескриптора (растёт с σ), **отрезок** — ориентация θ.

Изменение	Какой шаг с ним справляется
сдвиг	точку ищем по содержимому, а не по координатам
масштаб	экстремум по σ (шаги 1–3), окно $\propto \sigma$
поворот	ориентация θ (шаг 4)
яркость $I + b$	используем только градиенты
контраст aI	нормировка вектора
нелинейный свет	отсечение 0.2
ракурс, шум	гистограммы по ячейкам **терпят** мелкие сдвиги

Не инвариантен: сильная перспектива (больше $\sim 40^\circ$ ракурса), изменение самой сцены, повторяющиеся узоры (окна, плитка).

Листинг: SIFT и сопоставление в OpenCV

```python
import cv2

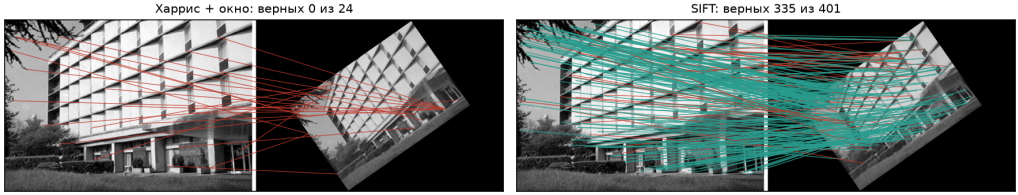
sift = cv2.SIFT_create()
kA, dA = sift.detectAndCompute(A, None)
kB, dB = sift.detectAndCompute(B, None)
# kA[i].pt, .size, .angle
# dA: массив (N, 128), float32

bf = cv2.BFMatcher(cv2.NORM_L2)
knn = bf.knnMatch(dA, dB, k=2)
good = [m for m, n in knn
        if m.distance < 0.75 * n.distance]

src = [kA[m.queryIdx].pt for m in good]
dst = [kB[m.trainIdx].pt for m in good]
```

- `A`, `B` — `uint8`, оттенки серого; `None` — без маски.
- `.size` — диаметр области ($\propto \sigma$), `.angle` — θ в градусах.
- `knnMatch(k=2)` даёт **два** ближайших дескриптора B для каждого из A — нужны для ratio test (часть 6).
- `queryIdx` — номер точки в A, `trainIdx` — в B.
- Пары `src`, `dst` пойдут в `cv2.findHomography(..., cv2.RANSAC)` — лекция 7.

Харрис и SIFT на одной паре



Метод	точек (A / B)	время на кадр	пар после ratio 0.8	из них верных
Харрис + окно 11×11	573 / 252	≈ 7 мс	24	0
SIFT	1000 / 1000	≈ 75 мс	401	335 (84 %)

Второй снимок повёрнут на 35° и уменьшен до 0.6; 640×442, один поток CPU, OpenCV 5.0. Харрис без масштаба и ориентации ломается полностью: его окно не поворачивается и не растягивается вместе с объектом.

Быстрые альтернативы SIFT (для справки)

SIFT тратит десятки миллисекунд на кадр. Для видео и телефонов придуманы более быстрые методы; подробно мы их не разбираем, но названия стоит знать.

Метод	**Идея в одной строке**
SURF (2006)	SIFT-подобный, но фильтры из прямоугольников — быстрее за счёт интегрального изображения
FAST (2006)	очень быстрый детектор углов: сравнение пикселя с соседями на окружности
BRIEF (2010)	двоичный дескриптор из попарных сравнений яркости, сравнивается расстоянием Хэмминга
ORB (2011)	FAST + ориентация + BRIEF: на порядок быстрее SIFT, но менее точен; есть в OpenCV

Конвейер тот же: детектор → дескриптор → сопоставление с ratio test.

Часть 5

Сопоставление и оценка

О чём эта часть

Определение

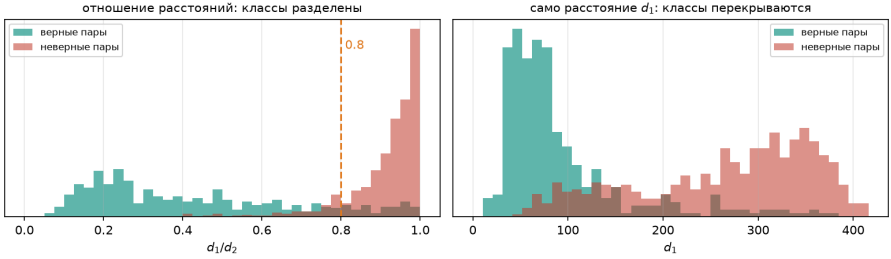
Ближайший сосед (NN) — для дескриптора a первого снимка дескриптор b_1 второго с минимальным расстоянием $d_1 = \|a - b_1\|$.

Ratio test (Lowe) — принимать пару, только если ближайший сосед **заметно ближе второго**: $d_1/d_2 < 0.8$.

Что покажем

- почему порог на d_1 плохо работает;
- ratio test и взаимная проверка;
- быстрый поиск: kd-дерево, FLANN, LSH;
- метрики: повторяемость, matching score, precision–recall.

Почему отношение расстояний



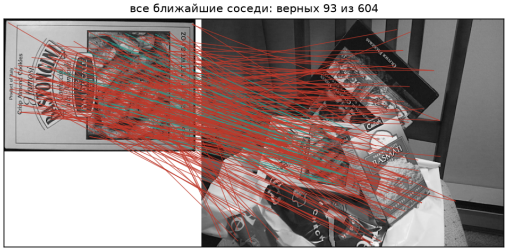
принять пару (a, b_1), если $\dfrac{d_1}{d_2} < 0.8$

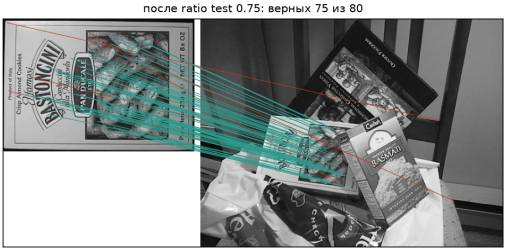
- d_1, d_2 — расстояние до первого и второго ближайшего дескриптора на другом снимке;
- 0.8 — порог Лоу: отбрасывает около 90 % неверных пар и меньше 5 % верных.

Почему работает. У точки, которой **нет** на втором снимке, все кандидаты случайны — $d_1 \approx d_2$, отношение около 1. У верной пары d_1 **намного меньше** расстояния до любого постороннего. А сами d_1 сильно разные у разных точек (текстура, контраст): единый порог по d_1 не подходит.

Данные: SIFT, поворот 35° и масштаб 0.6; 382 верных и 618 неверных ближайших соседей.

Ratio test на практике





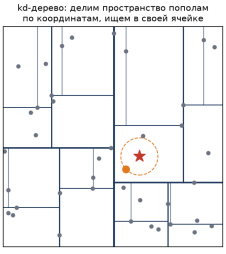
Фильтр	пар	верных	точность
ближайший сосед	604	93	15 %
взаимная проверка	260	88	34 %
ratio 0.75	80	75	94 %
ratio + взаимная	77	73	95 %

Коробка ищется на столе с другими предметами: у большинства точек сцены **нет пары** на образце.

Взаимная проверка (cross-check): b — ближайший к a, **и** a — ближайший к b. Работает без второго соседа, но слабее ratio test. Вместе — чуть чище.

Верность — по гомографии, найденной RANSAC (лекция 7).

Быстрый поиск ближайшего соседа





Перебор: $N_A \times N_B$ расстояний. Для 1000×1000 SIFT — $10^6 \cdot 128$ операций, ≈ 10 мс; для базы в миллион изображений — недопустимо.

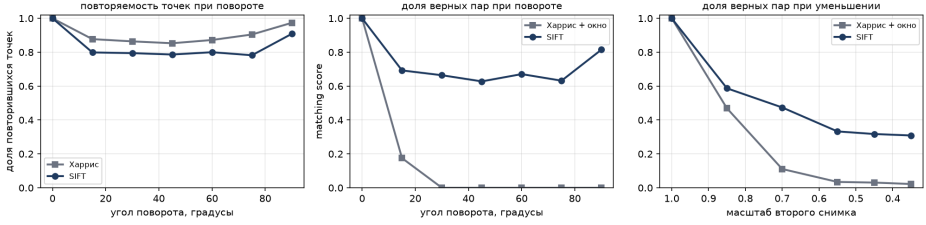
kd-дерево: делим пространство пополам по координатам; запрос спускается в свою ячейку. В 128 измерениях точный поиск почти не ускоряется, поэтому ищут **приближённо**: смотрят ограниченное число ячеек.

FLANN (Muja, Lowe, 2009): несколько случайных kd-деревьев или иерархический k-means.

LSH (хеширование с учётом близости): случайные гиперплоскости дают вектору короткий двоичный код; кандидаты — векторы **с тем же кодом**, среди них считаем точное расстояние.

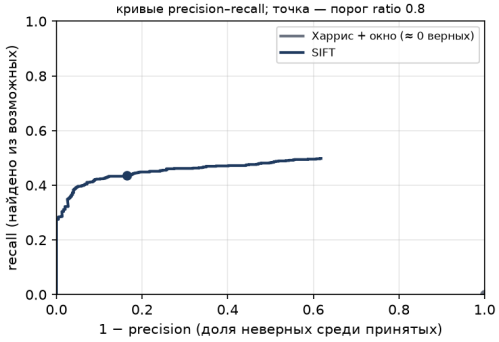
$$\text{rep} = \frac{\#\{\text{точки, найденные на обоих снимках}\}}{\min(N_A,\ N_B)}, \qquad \text{MS} = \frac{\#\{\text{верные пары}\}}{\min(N_A,\ N_B)}$$

- точка «найдена на обоих», если её образ Hx при известном преобразовании H попал ближе ε (здесь 3 пикселя) к точке второго снимка; N_A, N_B — число точек **в общей части** снимков;
- **rep** (repeatability) — качество **детектора**; **MS** (matching score) — детектора **и дескриптора** вместе.



Оба детектора **повторяются при повороте**: точки Харриса находятся снова в 85–97 % случаев. Но Харрис с окном 11×11 не **сопоставляется** уже при 15°: окно-дескриптор не поворачивается. При уменьшении SIFT держит ≈ 0.3 даже на масштабе 0.35, Харрис падает почти до нуля уже на 0.55.

Кривые precision–recall



$$P = \frac{\#\text{верных}}{\#\text{принятых}}, \qquad R = \frac{\#\text{верных}}{\#\text{возможных}}$$

- P — precision (точность), R — recall (полнота); принятые — пары, прошедшие порог ratio; возможные — точки, у которых на втором снимке **есть** точка-образ;
- меняя порог от 0 до 1, получаем кривую: чем **левее и выше**, тем лучше.

На паре «поворот 35° + масштаб 0.6» SIFT при пороге 0.8 находит около 44 % возможных пар при 16 % ошибок; Харрис с окном — около нуля.

Бенчмарки: Oxford Affine (2005; отсюда граффити) и HPatches (2017) — на них сравнивают и нейросети.

Листинг: FLANN и взаимная проверка

```
# приближённый поиск: kd-деревья
idx = dict(algorithm=1, trees=5)
search = dict(checks=50)
fl = cv2.FlannBasedMatcher(idx, search)
knn = fl.knnMatch(dA, dB, k=2)
good = [m for m, n in knn
        if m.distance < 0.8 * n.distance]

# взаимная проверка (точный перебор)
bf = cv2.BFMatcher(cv2.NORM_L2,
                   crossCheck=True)
pairs = bf.match(dA, dB)
```

- `dA`, `dB` — дескрипторы SIFT (`float32`, $N \times 128$).
- `algorithm=1` — индекс из случайных kd-деревьев, `trees=5` — сколько деревьев.
- `checks=50` — сколько листьев просмотреть: больше — точнее и медленнее.
- `crossCheck=True` — взаимная проверка прямо в `match`; вместе с ней `knnMatch` не используют.
- Для тысяч точек хватает перебора; FLANN нужен при **десятках тысяч** и больше.

Мост к следующему семестру

Где эта идея живёт в глубоком обучении

- **SuperPoint** (DeTone и др., 2018) и **DISK** (Tyszkiewicz и др., 2020) учат детектор и дескриптор одной сетью. Обучающий сигнал — **та же повторяемость**: точка должна находиться снова на искусственно искажённой (гомографией) копии снимка.
- Выход сети — карта «где точка» плюс карта дескрипторов; дальше — подавление немаксимумов, ближайший сосед и **ratio test**, как сегодня.
- **SuperGlue** (2020), **LightGlue** (2023) заменяют ratio test обучаемым сопоставлением с вниманием; **LoFTR** (2021) обходится без детектора вовсе.
- Оценивают их на тех же **HPatches** и по тем же метрикам: повторяемость, matching score, точность гомографии.

Подробно — в курсе «Глубокое обучение в задачах компьютерного зрения». Здесь только связь, без деталей.

Итоги лекции

1. Особые точки нужны для сопоставления снимков; требования — повторяемость, различимость, локальность, эффективность.
2. Сдвиг окна $E(u,v) \approx (u,v)\, M\, (u,v)^\top$; собственные числа структурного тензора M: обе малы — плоско, одна — край, обе велики — **угол**.
3. Харрис $R = \det M - k\,\mathrm{tr}^2 M$, Ши–Томаси $R = \min(\lambda_1, \lambda_2)$; инвариантны к повороту, **не к масштабу**.
4. Масштаб выбирают по максимуму $\sigma^2 \nabla^2 L$ по σ; для пятна радиуса r — $\sigma^* \approx r/\sqrt{2}$; DoG уже нормирован.
5. SIFT: пирамида DoG $\to$ экстремумы $3 \times 3 \times 3$ $\to$ уточнение, отсев контраста и краёв ($r = 10$) $\to$ ориентация $\to$ $4 \times 4 \times 8 = 128$ с нормировкой и отсечением 0.2.
6. Харрис без масштаба и ориентации на паре «поворот + уменьшение» не даёт ни одной верной пары; SIFT — 84 % верных.
7. Сопоставление: ближайший сосед + **ratio test** $d_1/d_2 < 0.8$ + взаимная проверка; оценка — повторяемость, matching score, precision–recall.

Литература

Основная

Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., гл. 7.1 (Points and patches) — `https://szeliski.org/Book/`.
Gonzalez, Woods, *Digital Image Processing*, 4th ed., гл. 11 (Feature extraction: SIFT).

Первоисточники

Harris, Stephens, *A combined corner and edge detector*, Alvey Vision Conference, 1988.
Shi, Tomasi, *Good features to track*, CVPR, 1994.
Lindeberg, *Feature detection with automatic scale selection*, IJCV, 1998.
Lowe, *Distinctive image features from scale-invariant keypoints*, IJCV, 2004.
Mikolajczyk, Schmid, *A performance evaluation of local descriptors*, IEEE TPAMI, 2005.
Снимки здания, коробки, граффити и гомография `H1to3p` — OpenCV `samples/data` (лицензия Apache 2.0); остальное — `skimage.data`; иллюстрации — `scripts/make_figs_L06.py`.

Вопросы для самопроверки

1. Какие четыре требования предъявляют к особым точкам? Почему точка на границе плохая?
2. Выведите $E(u,v) \approx (u,v)\,M\,(u,v)^\top$. Что лежит в матрице M?
3. Как по собственным числам M отличить плоский участок, край и угол? Запишите отклики Харриса и Ши–Томаси.
4. Почему Харрис инвариантен к повороту и не инвариантен к масштабу?
5. Зачем нормировать LoG на σ^2? Какой масштаб выберется для пятна радиуса r?
6. Опишите шаги SIFT. Какой шаг даёт инвариантность к масштабу, повороту, контрасту?
7. Как SIFT отбрасывает точки на краях? Выведите $\operatorname{tr}^2 H / \det H = (r+1)^2/r$.
8. Как SIFT находит доминирующую ориентацию и зачем она нужна?
9. Как устроен дескриптор SIFT? Зачем нормировка и отсечение 0.2?
10. Что такое ratio test и почему он лучше порога на d_1? Как измеряют repeatability и matching score?